

GRUPO BRAVO

Restaurante Bravo · Plataforma full-stack

Propuesta senior de nuevas funcionalidades

Backlog priorizado · Estimación de horas · Roadmap

Audiencia: equipo de desarrollo

Versión 1.0 · 5 de mayo de 2026

1. Resumen ejecutivo

Este documento recoge la investigación senior de la plataforma GRUPO BRAVO (Restaurante Bravo) y el backlog priorizado de nuevas funcionalidades para los próximos trimestres. La revisión cubre las cuatro capas del producto: backend (FastAPI + MongoDB), frontend (Flutter multiplataforma), base de datos y DevOps/infraestructura.

La aplicación ya cubre la operativa básica: gestión multi-restaurante, pedidos en local/domicilio/recoger, reservas, mesas con QR, ingredientes y stock, cupones, autenticación con JWT y 2FA, pagos integrados con Stripe, PayPal, Apple Pay y Google Pay, así como auditoría y cumplimiento RGPD/PCI-DSS. Sobre esa base sólida, este documento propone las próximas iteraciones de producto y plataforma.

Cifras clave

Funcionalidades propuestas	38
Horas estimadas totales	1049 h
Equivalente en jornadas (8 h)	131 días
Equivalente en sprints (1 dev × 2 sem)	≈ 13.1 sprints
Equipo recomendado	2 backend · 2 frontend · 1 DevOps · 1 PO
Duración (3 devs en paralelo, sprints de 2 sem)	≈ 5 sprints (~10 semanas)

Cómo leer este documento

- La sección 4 contiene la tabla maestra de tareas, ordenadas globalmente de la más fácil a la más difícil, con su estimación en horas.
- Las secciones 5 a 7 detallan cada tarea agrupada por nivel de complejidad: Fácil, Media y Difícil.
- La sección 8 propone un roadmap de sprints. La sección 9 recoge riesgos y dependencias técnicas.

2. Contexto del producto

Stack actual detectado

- Backend: FastAPI 0.115, Python 3.11+, MongoDB 7, JWT (python-jose), bcrypt, Pydantic 2, slowapi (rate limiting), pyotp (2FA TOTP), fastapi-mail, Stripe 11, qrcode, mongomock para tests.
- Frontend: Flutter 3.27+, Provider, http, mobile_scanner (QR), qr_flutter, flutter_stripe, geolocator, flutter_map, flutter_local_notifications, audioplayers, cached_network_image, google_fonts.
- Operativa: multi-restaurante con super_admin, roles cliente / camarero / cocinero / repartidor / admin / super_admin, pedidos local/domicilio/recoger, reservas con detección de conflictos, tickets de mesa, cupones, ingredientes y stock con descuento atómico, auditoría dual (acciones y pagos), exportación y baja RGPD.
- Calidad: 66+ tests backend con mongomock y 100+ tests frontend, CI con GitHub Actions, dart format / flutter analyze obligatorios, log_redactor PCI-DSS, scripts check-ci.sh, healthchecks Docker, init_indexes.py para MongoDB.

Áreas con margen claro de mejora

- Engagement de cliente: fidelización, reseñas, recomendaciones, push notifications.
- Operativa avanzada: split bill, refunds, depósitos en reservas, modo kiosko, impresora térmica.
- Plataforma: observabilidad, cache distribuida, pruebas E2E, backups automatizados.

- Crecimiento: integraciones con plataformas delivery, exportación contable, multi-moneda.

3. Metodología y criterios de estimación

Las estimaciones asumen un desarrollador senior con conocimiento del stack y dedicación a tiempo completo. Cada estimación incluye análisis, implementación, tests, documentación y revisión por pares (PR review). No incluye QA externo, traducciones profesionales ni acuerdos comerciales con terceros (delivery, TPV, contabilidad).

Niveles de complejidad

Nivel	Tareas	Horas	Tipo de trabajo
Fácil	8	59 h	Tareas autónomas, bajo riesgo, valor visible al usuario en pocos días.
Media	16	352 h	Requieren coordinación frontend-backend o cambios de modelo. Ideal para sprints de 2 semanas.
Difícil	14	638 h	Integraciones externas, cambios estructurales, observabilidad y plataforma. Riesgo medio-alto.

Criterios de priorización (orden global)

- 1. Esfuerzo y riesgo técnico (lo más sencillo y aislado primero).
- 2. Independencia de terceros (Stripe ya está; FCM, partners delivery, contabilidad llegan después).
- 3. Reutilización de modelos y endpoints existentes.
- 4. Impacto sobre el cliente final dentro del mismo nivel.

4. Tabla maestra de tareas

Ordenadas globalmente de la más fácil a la más difícil. El identificador (F-, M- o D-) refleja el nivel; la columna «Horas» es la estimación de un único desarrollador.

ID	Tarea / Funcionalidad	Área	Nivel	Horas	Dependencias
F-01	Modo oscuro / theme toggle	Frontend	Fácil	5 h	—
F-02	Etiquetas en productos (popular, nuevo, oferta)	Frontend	Fácil	6 h	—
F-03	Búsqueda y filtros básicos en la carta	Frontend	Fácil	8 h	—
F-04	Información de alérgenos en producto	Backend+FE	Fácil	7 h	—
F-05	Onboarding tutorial breve (cliente)	Frontend	Fácil	8 h	—

F-06	Repetir pedido desde el historial	Frontend	Fácil	8 h	—
F-07	Filtros y ordenación en historial de pedidos	Backend+FE	Fácil	9 h	—
F-08	Propinas en el flujo de pago	Backend+FE	Fácil	8 h	Stripe (existente)
M-09	Internacionalización (ES/EN/CA)	Frontend	Media	18 h	—
M-10	Reseñas y valoraciones de productos y restaurante	Backend+FE	Media	24 h	Pedidos pagados
M-11	Exportación de pedidos y ventas (CSV/Excel)	Backend+FE	Media	14 h	—
M-12	Programa de fidelización (puntos)	Backend+FE	Media	28 h	Pedidos pagados
M-13	Pre-orden / programar hora de entrega	Backend+FE	Media	18 h	—
M-14	Encuestas post-pedido y NPS	Frontend	Media	18 h	Pedidos pagados
M-15	Importación masiva de productos (CSV/Excel)	Backend+FE	Media	16 h	—
M-16	Modo offline básico (cache de menú)	Frontend	Media	22 h	—
M-17	Recomendaciones «más vendidos» y «te puede interesar»	Backend+FE	Media	20 h	Histórico pedidos
M-18	Notificaciones push remotas (FCM)	Frontend	Media	26 h	Backend con worker o cron
M-19	Reservas con depósito reembolsable	Backend+FE	Media	26 h	Stripe
M-20	Lista de espera (waitlist) sin mesa	Backend+FE	Media	18 h	—
M-21	Refunds y cancelaciones con reembolso Stripe	Backend+FE	Media	22 h	Stripe
M-22	Dividir cuenta (split bill)	Backend+FE	Media	26 h	Tickets de mesa, Stripe
M-23	Tracking del repartidor en mapa (cliente)	Backend+FE	Media	26 h	Geolocator (existente)
M-24	Modo kiosko de auto-servicio	Backend+FE	Media	30 h	Pagos
D-25	Dashboard analítico con KPIs y gráficas	Backend+FE	Difícil	50 h	Histórico pedidos
D-26	Gift cards / tarjetas regalo	Backend+FE	Difícil	44 h	Stripe
D-27	Pedido recurrente / suscripción semanal	Backend+FE	Difícil	46 h	Stripe Subscriptions, Pre-orden
D-28	Integración con plataformas delivery (Glovo / Uber Eats / JustEat)	Backend	Difícil	70 h	API de cada partner
D-29	Chat / soporte interno cliente ↔	Backend+FE	Difícil	50 h	Push notifications

	camarero				
D-30	Soporte multi-moneda y multi-impuestos	Backend+FE	Difícil	40 h	Restaurantes con moneda
D-31	Integración con impresora térmica (cocina y ticket)	Frontend	Difícil	36 h	Tickets
D-32	Webhooks de salida y eventos a sistemas externos	Backend	Difícil	30 h	—
D-33	Cache distribuida con Redis para menú/carta	DevOps	Difícil	28 h	Despliegue
D-34	Observabilidad: Sentry + structured logs + métricas	DevOps	Difícil	32 h	—
D-35	Pruebas E2E (Playwright + Flutter integration_test)	DevOps	Difícil	50 h	CI verde
D-36	Backup automatizado y plan de recuperación (DR)	DevOps	Difícil	32 h	—
D-37	Exportación contable (Holded / Quaderno / Sage)	Backend	Difícil	50 h	Multi-impuestos
D-38	Integración con TPVs externos (Square, SumUp, REDSYS)	Backend+FE	Difícil	80 h	Hardware partner
	TOTAL ESTIMADO		38 tareas	1049 h	≈ 13.1 sprints

5. Nivel Fácil — 8 tareas (59 h)

Tareas con bajo riesgo, alcance acotado y feedback rápido al usuario. Adecuadas para cualquier developer del equipo y para validar nuevos integrantes.

F-01 · Modo oscuro / theme toggle (5 h · Frontend)

Interruptor manual (claro/oscuro/sistema) en perfil cliente y trabajador. Persistencia con shared_preferences. Mantener tokens de color en core/colors_style.dart y app_theme.dart, asegurando contraste AA en cocina (alto brillo).

Archivos / módulos: core/app_theme.dart, perfil_screen.dart, splash, todos los Scaffold.

Depende de: —

F-02 · Etiquetas en productos (popular, nuevo, oferta) (6 h · Frontend)

Mostrar chips de estado en producto_card.dart («Nuevo», «Más pedido», «-20%»). Backend devuelve un campo etiquetas: list[str] en GET /productos. Admin puede activar manualmente desde admin_editar_plato.

Archivos / módulos: models/producto_model.dart, producto_card.dart, routes/productos.py, admin_editar_plato.dart.

Depende de: —

F-03 · Búsqueda y filtros básicos en la carta (8 h · Frontend)

TextField con debounce + chips (categoría, vegetariano, sin gluten, rango de precio) en `carta_screen.dart`. Lógica solo cliente; el backend ya devuelve el listado completo por sucursal.

Archivos / módulos: `carta_screen.dart`, `producto_model.dart`.

Depende de: —

F-04 · Información de alérgenos en producto (7 h · Backend+FE)

Añadir campo alergen: `list[str]` en `producto_model` y modelo Pydantic. Iconografía estándar UE-1169/2011 (gluten, lactosa, frutos secos, etc.) en `producto_detalle_sheet.dart`. Filtro «sin alérgenos X» en la carta.

Archivos / módulos: `models.py`, `productos.py`, `producto_model.dart`, `producto_detalle_sheet.dart`.

Depende de: —

F-05 · Onboarding tutorial breve (cliente) (8 h · Frontend)

3 pantallas de bienvenida la primera vez tras login del cliente: «Pide en mesa con QR», «Reserva», «Pedidos a domicilio». Skipable. Marca un flag en `SharedPreferences` para no repetir.

Archivos / módulos: `Nuevas screens en cliente/onboarding/`, `hook en main.dart`.

Depende de: —

F-06 · Repetir pedido desde el historial (8 h · Frontend)

Botón «Volver a pedir» en `historial_pedidos_screen.dart` que recarga los items disponibles al carrito y abre `opciones_entrega_screen`. Aviso si algún producto ya no existe o está sin stock.

Archivos / módulos: `historial_pedidos_screen.dart`, `cart_provider.dart`, `pedido_service.dart`.

Depende de: —

F-07 · Filtros y ordenación en historial de pedidos (9 h · Backend+FE)

Parámetros desde/hasta, estado, tipoEntrega y orden ASC/DESC en GET /pedidos. UI con date-range picker y chips. Útil tanto para cliente como camarero.

Archivos / módulos: `routes/pedidos.py`, `historial_pedidos_screen.dart`, `gestion_pedidos.dart`.

Depende de: —

F-08 · Propinas en el flujo de pago (8 h · Backend+FE)

Selector de propina (0% / 5% / 10% / personalizado) antes del `PaymentIntent`. Sumar al amount enviado a Stripe y guardarla como propina en el pedido. Mostrar línea separada en factura HTML/email.

Archivos / módulos: `models.py` (`PedidoCrear`), `pagos.py`, `opciones_entrega_screen.dart`, `_enviar_factura()`.

Depende de: Stripe (existente)

6. Nivel Media — 16 tareas (352 h)

Tareas que requieren coordinación entre frontend, backend y, en algunos casos, modificaciones en el modelo de datos. Encajan bien en sprints de dos semanas con un desarrollador full-stack o en pareja.

M-09 · Internacionalización (ES/EN/CA) (18 h · Frontend)

Migrar strings hardcoded a `flutter_localizations` + ARB. Selector de idioma en perfil. Cobertura mínima: cliente y trabajador. Auditoría con `grep` para localizar strings literales.

Archivos / módulos: *Todo lib/ con literales en español. Nuevo l10n/.*

Depende de: —

M-10 · Reseñas y valoraciones de productos y restaurante (24 h · Backend+FE)

Nuevo recurso /api/v1/resenas con valoración (1-5), comentario y referencia a pedido_id. Solo se permite reseñar productos del propio pedido entregado. Promedio cacheado en producto y restaurante. Moderación por admin.

Archivos / módulos: *Nuevo routes/resenas.py, modelo Resena, pantalla resena_screen.dart.*

Depende de: Pedidos pagados

M-11 · Exportación de pedidos y ventas (CSV/Excel) (14 h · Backend+FE)

Endpoint admin GET /pedidos/exportar?desde=&hasta=&formato=csv|xlsx. Generación con openpyxl en backend. Botón «Exportar» en admin_contabilidad_screen.dart. Incluir totales, propinas, IVA y método de pago.

Archivos / módulos: *Nuevo backend/services/exportador.py, admin_contabilidad_screen.dart.*

Depende de: —

M-12 · Programa de fidelización (puntos) (28 h · Backend+FE)

1 punto = 1 € gastado. Canje en checkout (X puntos = Y € de descuento). Transacciones en colección puntos_movimientos para trazabilidad. UI: tarjeta de saldo en perfil cliente y selector en checkout.

Archivos / módulos: *Nueva colección, routes/fidelizacion.py, perfil_screen.dart, opciones_entrega_screen.dart.*

Depende de: Pedidos pagados

M-13 · Pre-orden / programar hora de entrega (18 h · Backend+FE)

Selector «Lo antes posible» / «Programar para hh:mm» en checkout. Validación contra horario de la sucursal (restaurantes.horario). Backend: campo fecha_programada y cola de pedidos futuros para cocina.

Archivos / módulos: *PedidoCrear, routes/pedidos.py, opciones_entrega_screen.dart, vista cocinero.*

Depende de: —

M-14 · Encuestas post-pedido y NPS (18 h · Frontend)

Tras estado=entregado, enviar push o email con enlace a encuesta corta (rapidez, sabor, atención, NPS 0-10). Backend almacena resultados y dashboard agrega media.

Archivos / módulos: *Nueva colección encuestas, plantilla email, pantalla encuesta_screen.dart.*

Depende de: Pedidos pagados

M-15 · Importación masiva de productos (CSV/Excel) (16 h · Backend+FE)

Pantalla en super_admin con drag & drop. Backend parsea con openpyxl y valida campos contra ProductoCrear. Retorna informe de filas insertadas / con error. Log en auditoria.

Archivos / módulos: *catalogo_masivo_screen.dart, nuevo endpoint POST /productos/import.*

Depende de: —

M-16 · Modo offline básico (cache de menú) (22 h · Frontend)

Cache de productos y categorías por sucursal con sqflite (móvil) e IndexedDB (web). Serve-stale-while-revalidate al abrir la carta. Indicador visual de «datos no actualizados» si offline.

Archivos / módulos: *services/api_service.dart, nueva capa cache, carta_screen.dart.*

Depende de: —

M-17 · Recomendaciones «más vendidos» y «te puede interesar» (20 h · Backend+FE)

Backend agrega top 10 productos por sucursal últimos 30 días. «Te puede interesar»: ítems pedidos junto al actual (co-ocurrencia simple). Carruseles en inicio_screen.dart y producto_detalle.

Archivos / módulos: *Nuevo endpoint /productos/populares, /productos/{id}/relacionados; widgets carrusel.*

Depende de: Histórico pedidos

M-18 · Notificaciones push remotas (FCM) (26 h · Frontend)

Integrar firebase_messaging para Android, APNs para iOS, web push para web. Backend con SDK admin envía: «pedido listo», «pedido en reparto», «oferta nueva». Token registrado en usuarios.fcm_tokens.

Archivos / módulos: *pubspec.yaml, services/notificaciones_service.dart, nuevo backend/services/push.py.*

Depende de: Backend con worker o cron

M-19 · Reservas con depósito reembolsable (26 h · Backend+FE)

Configuración por sucursal: depósito X € si la reserva es $\geq N$ comensales. Stripe authorize (manual capture) al crear la reserva. Captura/cancel automáticos al finalizar/cancelar. Política de cancelación (T-24h, T-2h).

Archivos / módulos: *ReservaCrear, routes/reservas.py, pagos.py (Stripe holds).*

Depende de: Stripe

M-20 · Lista de espera (waitlist) sin mesa (18 h · Backend+FE)

Cuando todas las mesas están ocupadas, botón «Apuntarme a la lista» con nº de comensales. Camarero lo ve en `gestion_reservas.dart` y notifica al cliente cuando hay mesa.

Archivos / módulos: *Nueva colección waitlist, routes/waitlist.py, vista camarero.*

Depende de: —

M-21 · Refunds y cancelaciones con reembolso Stripe (22 h · Backend+FE)

Endpoint POST `/pagos/{pedido_id}/refund` (admin/super_admin) con motivo. Llamada a `stripe.Refund.create`. Estado del pedido pasa a reembolsado. Auditoría en `auditoria_pagos`. Vista admin con motivo y porcentaje.

Archivos / módulos: *pagos.py, gestion_pedidos.dart (rol admin), models.py.*

Depende de: Stripe

M-22 · Dividir cuenta (split bill) (26 h · Backend+FE)

Split por igual o por ítems desde `sacar_cuenta.dart`. Cada comensal escanea QR de pago para pagar su parte (Stripe Checkout). Cierra ticket cuando todas las partes están pagadas.

Archivos / módulos: *tickets.py, pagos.py, sacar_cuenta.dart, nueva pantalla pago_split.dart.*

Depende de: Tickets de mesa, Stripe

M-23 · Tracking del repartidor en mapa (cliente) (26 h · Backend+FE)

Repartidor envía ubicación cada 15 s (background). Cliente ve en mapa con `flutter_map` la posición y ETA. Backend guarda traza ligada al pedido en `repartidor_tracking`. Borrar al entregar.

Archivos / módulos: *Nuevo endpoint POST `/pedidos/{id}/tracking`, websocket o polling, pantalla cliente.*

Depende de: Geolocator (existente)

M-24 · Modo kiosko de auto-servicio (30 h · Backend+FE)

Pantalla pensada para tablet en local sin login: elegir mesa o «para llevar», carta táctil, pago con tarjeta, ticket impreso. Bloqueo de salida (modo quiosco). Auto-logout y reset al inactivar.

Archivos / módulos: *Nueva ruta `/kiosko`, layout específico, integración con impresora.*

Depende de: Pagos

7. Nivel Difícil — 14 tareas (638 h)

Tareas con integraciones externas, cambios estructurales o requisitos de plataforma (observabilidad, cache, backups). Requieren diseño previo y, en varios casos, acuerdos comerciales con terceros.

D-25 · Dashboard analítico con KPIs y gráficas (50 h · Backend+FE)

Vista admin/super_admin con: ventas/día, ticket medio, productos top, hora pico, tasa cancelación, ocupación de mesas, NPS. Backend con pipelines de aggregate. Frontend con `fl_chart` o `sparkline`. Filtros por sucursal, fecha, método de pago.

Archivos / módulos: *Nuevo `routes/analytics.py`, `admin_contabilidad` y `kpis_globales_screen`, librería de gráficas.*

Depende de: Histórico pedidos

D-26 · Gift cards / tarjetas regalo (44 h · Backend+FE)

Cliente compra una gift card por importe X. Recibe código por email. El código se canjea en checkout (descuenta saldo). Multi-uso hasta agotar saldo. Conciliación contable como ingreso diferido.

Archivos / módulos: Nueva colección `gift_cards`, `routes/giftcards.py`, `plantilla email`, `pantallas`.

Depende de: Stripe

D-27 · Pedido recurrente / suscripción semanal (46 h · Backend+FE)

Cliente programa repetición (lun y mié, mismo carrito, misma dirección). Backend con APScheduler genera pedidos automáticamente. Stripe Subscription para cobro periódico. Pausa, cancelación, modificación.

Archivos / módulos: Cron interno o `sidecar`, `routes/suscripciones.py`, `perfil_screen.dart`.

Depende de: Stripe Subscriptions, Pre-orden

D-28 · Integración con plataformas delivery (Glovo / Uber Eats / JustEat) (70 h · Backend)

Sincronización del menú a cada plataforma. Recepción de pedidos vía webhook y normalización al modelo PedidoCrear. Actualización de estado (aceptado/rechazado, tiempos). Conciliación contable diferenciada.

Archivos / módulos: Nuevo paquete `backend/integrations/`, `mappers`, `gestion_pedidos`.

Depende de: API de cada partner

D-29 · Chat / soporte interno cliente ↔ camarero (50 h · Backend+FE)

Canal de mensajería ligada al pedido («Falta la salsa», «¿Cuánto falta?»). WebSocket con FastAPI o Pusher/Ably. Persistencia en Mongo. UI conversación tipo bandeja en `gestion_pedidos`.

Archivos / módulos: Nuevo módulo `backend/ws/`, `services/chat.dart`, `pantallas chat`.

Depende de: Push notifications

D-30 · Soporte multi-moneda y multi-impuestos (40 h · Backend+FE)

Campo moneda y impuesto (IVA, IGIC, etc.) por sucursal. Conversión visual con tipos de cambio diarios (API gratuita). Stripe en moneda nativa de la sucursal. Etiquetado en facturas y reports.

Archivos / módulos: `restaurantes`, `productos`, `pedidos`, `pagos.py`, `formatters de UI`.

Depende de: Restaurantes con moneda

D-31 · Integración con impresora térmica (cocina y ticket) (36 h · Frontend)

Drivers ESC/POS para Bluetooth, USB y red (Epson TM-T20, Star TSP). Plantilla configurable: encabezado, items, total, QR de reseña. Cola con reintento si la impresora está apagada.

Archivos / módulos: `core/printing_helper`, `nueva configuración por sucursal`, `sacar_cuenta.dart`.

Depende de: Tickets

D-32 · Webhooks de salida y eventos a sistemas externos (30 h · Backend)

Configuración por sucursal: URL + secreto + eventos suscritos (`pedido.creado`, `pedido.entregado`, `pago.completado`). Firma HMAC, reintento exponencial, `dead-letter queue`.

Archivos / módulos: Nuevo `backend/services/webhooks_out.py`, `admin_local_screen.dart`.

Depende de: —

D-33 · Cache distribuida con Redis para menú/carta (28 h · DevOps)

Capa de cache para GET `/productos` y `/categorias` por restaurante (TTL 5 min, invalidación al editar). Redis containerizado, fallback transparente si no está. Reduce QPS contra Mongo en horas pico.

Archivos / módulos: `docker-compose.yml`, nuevo `backend/services/cache.py`, `routes afectadas`.

Depende de: Despliegue

D-34 · Observabilidad: Sentry + structured logs + métricas (32 h · DevOps)

Sentry SDK en backend y Flutter. Logs JSON con request_id (correlación). Endpoint /metrics estilo Prometheus (slowapi/uvicorn ya expone hooks). Alertas básicas (latencia p99, 5xx, fallos Stripe).

Archivos / módulos: *main.py, Dockerfile, frontend/lib/main.dart*.

Depende de: —

D-35 · Pruebas E2E (Playwright + Flutter integration_test) (50 h · DevOps)

Suite mínima de 8-10 escenarios críticos: registro+login, pedido en mesa, pedido a domicilio con pago, reserva, cancelación. CI con servicio Mongo efímero. Reporte HTML y screenshots en fallo.

Archivos / módulos: *Nuevo backend/tests_e2e/, frontend/integration_test/, .github/workflows/e2e.yml*.

Depende de: CI verde

D-36 · Backup automatizado y plan de recuperación (DR) (32 h · DevOps)

Backups diarios de MongoDB (mongodump cifrado a S3/B2). Retención 30 días, 12 meses, 7 años para los del 1.º de cada mes. Runbook de restore probado en staging. Cifrado en reposo y en tránsito.

Archivos / módulos: *Nuevo job cron, scripts/backup.sh, docs/RUNBOOK_DR.md*.

Depende de: —

D-37 · Exportación contable (Holed / Quaderno / Sage) (50 h · Backend)

Generación de asientos diarios y SII en formato compatible (CSV o API). Soporte facturas rectificativas para refunds. Flujo programado con confirmación humana antes de enviar.

Archivos / módulos: *Nuevo backend/integrations/contabilidad/, jobs nocturnos*.

Depende de: Multi-impuestos

D-38 · Integración con TPVs externos (Square, SumUp, REDSYS) (80 h · Backend+FE)

Aceptar tarjetas en datáfono físico vinculado al pedido. SDK del partner (Square Reader SDK, SumUp Plus). Sincronización de estados de transacción con auditoría dedicada.

Archivos / módulos: *Nuevo módulo de pagos por hardware, gestion_pedidos, configuración por sucursal*.

Depende de: Hardware partner

8. Roadmap propuesto por sprints

Sprints de dos semanas con tres desarrolladores trabajando en paralelo (1 backend, 1 frontend, 1 full-stack). Las dependencias técnicas se respetan en el orden de los sprints.

Sprint	Foco	Items	Horas
Sprint 1 (2 semanas)	Quick wins de UX cliente	F-01 Modo oscuro · F-02 Etiquetas en productos · F-03 Búsqueda y filtros · F-06 Repetir pedido	27 h
Sprint 2 (2 semanas)	Comercialización y pagos	F-04 Alérgenos · F-08 Propinas · F-07 Filtros pedidos · M-11 Exportación de ventas	38 h
Sprint 3 (2 semanas)	Engagement y fidelización	F-05 Onboarding · M-10 Reseñas · M-14 Encuestas NPS	50 h
Sprint 4 (2 semanas)	Crecimiento operativo	M-09 i18n · M-12 Fidelización con puntos · M-13 Pre-orden	64 h
Sprint 5 (2 semanas)	Operativa avanzada	M-15 Importación masiva · M-21 Refunds Stripe · M-	64 h

		19 Depósito reservas	
Sprint 6 (3 semanas)	Plataforma y observabilidad	D-34 Observabilidad · D-33 Cache Redis · M-16 Modo offline	82 h
Sprint 7-8 (4 semanas)	Analítica y push	D-25 Dashboard KPIs · M-18 Push remotas · M-17 Recomendaciones	96 h
Sprint 9-10 (4 semanas)	Diferenciadores y delivery	M-22 Split bill · M-23 Tracking repartidor · D-29 Chat soporte · D-31 Impresora térmica	138 h
Sprint 11+ (continuo)	Integraciones estratégicas	D-26 Gift cards · D-27 Pedidos recurrentes · D-28 Delivery 3rd party · D-37 Contabilidad · D-38 TPV externo	290 h

Hitos de negocio sugeridos

- Tras Sprint 2: el cliente percibe mejoras visibles (etiquetas, propinas, exportación de ventas).
- Tras Sprint 4: lanzamiento del programa de fidelización y pre-orden — campaña de marketing recomendada.
- Tras Sprint 6: plataforma con observabilidad y cache; preparada para escalar tráfico (Black Friday, festivos).
- Tras Sprint 8: dashboard analítico y push remotas — palancas de retención y upsell.
- Sprint 11+: diferenciadores comerciales (delivery, gift cards, contabilidad) según roadmap de negocio.

9. Riesgos y dependencias

9.1 Dependencias técnicas

- Stripe debe estar configurado en modo live con webhook firmado antes de propinas, depósitos, gift cards y suscripciones.
- Push notifications requieren cuenta Firebase y certificados APNs (Apple Developer Program de pago).
- Multi-moneda exige decisión de motor de cambios (BCE, OpenExchangeRates) y revisión jurídica.
- Integraciones delivery requieren acuerdos comerciales y revisión de cada API (rate limits, sandbox).

9.2 Riesgos a mitigar

- Deuda en gestión de strings: la i18n tardía obliga a una revisión completa de las pantallas.
- Ausencia de cache: el dashboard analítico puede impactar la base de datos si no se anticipa Redis o vistas materializadas.
- Coste de mantenimiento: cada integración externa (delivery, TPV, contabilidad) añade superficie de fallo y un canal de soporte.
- Cumplimiento: gift cards, multi-moneda y suscripciones tocan fiscalidad — coordinar con asesoría antes del lanzamiento.

9.3 Buenas prácticas recomendadas

- Activar feature flags por sucursal antes de lanzamientos generalizados (split bill, kiosko, delivery).
- Mantener cobertura de tests automatizados $\geq 80\%$ en backend y $\geq 60\%$ en frontend a medida que crece el alcance.
- Ejecutar test de carga (k6 o Locust) al menos antes de cada release mayor (post Sprint 6).
- Revisión de seguridad y pentest superficial al cerrar Sprint 8, antes de los diferenciadores comerciales.

10. Anexos

10.1 Backlog adicional fuera de alcance inicial

- Accesibilidad WCAG AA completa (auditoría + remediación) — 30-40 h.
- Modo «rush» en cocina (KDS) con tiempos de servicio y agrupación inteligente — 24-32 h.
- Firma digital del cliente al recibir el pedido a domicilio — 12-16 h.
- Plantillas de menú por temporada / horario lunch-dinner — 16-20 h.
- Sistema de invitaciones para staff (onboarding por email) — 12-16 h.
- Permisos granulares más allá de roles (por sucursal, por horario) — 24-30 h.

10.2 Glosario

- KDS: Kitchen Display System — pantalla de cocina con la cola de pedidos.
- ETA: Estimated Time of Arrival — tiempo estimado de entrega.
- FCM / APNs: servicios de notificaciones push de Google y Apple.
- ESC/POS: lenguaje de comandos estándar para impresoras térmicas.
- SII: Suministro Inmediato de Información (Hacienda España) para facturación.

10.3 Documentos relacionados

- Auditoria_Integral_GRUPO_BRAVO.docx — diagnóstico de seguridad, calidad y cumplimiento.
- Analisis_Arquitectura_GrupoBravo.docx — vulnerabilidades arquitectónicas y plan de remediación.
- Documentacion_Grupo_Bravo.docx — documentación técnica del sistema actual.
- docs/OPERACIONES_CRITICAS.md — checklist de acciones humanas (rotación de credenciales, webhook Stripe).
- docs/esquema_logico_db.md — esquema lógico de MongoDB.